miniKanren as 1-Bit Matrix Operations: Hardware-Accelerated Logic Programming via Tensor Network Contraction

L.J. Brian Cowan
loveJesus@loveJes.us

January 30, 2026

Abstract

We observe that *finite-domain* miniKanren constraint propagation reduces to sparse Boolean tensor operations. The key insight: when variable domains are finite bitmasks, unification becomes bitwise AND—a single CPU cycle. For structured terms, hash consing assigns integer IDs on demand, bridging infinite term languages to finite domain operations.

This mapping has three consequences. First, *parallelism*: domain intersection reduces to a single SIMD instruction. Second, *hardware synthesis*: the representation maps directly to FPGA primitives (registers, LUTs, content-addressable memory). Third, *differentiability*: Boolean operations generalize to semirings, enabling gradient-based learning.

Scope. This accelerates the constraint propagation *kernel*, not full miniKanren. Relational semantics (streams, fairness, occurs-check) are preserved via a reference implementation; the bit-parallel engine handles the hot inner loop.

We implement in Rust with FPGA targets (Calyx IR, Clash). Benchmarks show 3,000–4,000× speedup over heap-based operations on domain intersection, and 25–86× on N-Queens vs. Z3/clingo (all-solutions enumeration, domain-specific encoding). These speedups reflect representation advantages on finite combinatorial problems, not general SMT superiority.

Production-Scale Results (January 2026). We validated the approach on AWS F2 FPGA hardware (AMD VU47P-HBM) at 250 MHz. Using HBM batching for differentiable training, we achieve **3.3 million× speedup** for 1000-variable SAT training. The hardware implementation includes Q16.16 fixed-point Gumbel-softmax, enabling neural-guided SAT solving entirely in FPGA.

Hierarchical Domain Scaling. We compare hierarchical bitmap structures for large domains (262K values): 2-level 512^2 is **1.5× faster** than 3-level 64^3 (1.7 μs vs 2.5 μs intersection) due to fewer memory

indirections. The 512-bit words align with AVX-512 SIMD for further acceleration.

1 Introduction

Logic programming systems like Prolog and miniKanren [**?**] perform search over substitutions. Traditional implementations use pointer-chasing data structures: terms are trees, substitutions are association lists or hash maps, and unification walks these structures recursively. This is flexible but inherently sequential.

This paper explores a different representation. When variable domains are finite, we can represent them as bitmasks—bit i is 1 if value i is possible. Domain-equality constraints then become bitwise AND: the intersection of possible values. This is a single CPU instruction, trivially parallelizable.

We develop this observation into a complete system:

- **Domains as bitmasks**: A 64-bit word represents 64 possible values
- **Domain Unification as AND**: Constraint propagation via SIMD
- **Relations as tensors**: An n-ary relation is a sparse Boolean tensor
- **Composition as contraction**: Joining relations contracts shared dimensions
- **Infinite domains via hashing**: Terms are hash-consed to integer IDs on demand

The representation has practical consequences. On CPU, we achieve 3,000–4,000× speedup over heap-based approaches. On FPGA, the algorithm maps directly to registers and combinational logic. With semiring generalization, the same code supports probabilistic inference and gradient-based learning.

2 Background

2.1 miniKanren

miniKanren [**?**] is a relational programming language embedded in Scheme/Racket. Core operators:

- `(== t1 t2)` — unification goal
- `(conde [g1...] [g2...])` — disjunction (OR)
- `(fresh (x) g)` — introduce logic variable

- `(run n (q) g)` — find up to n solutions

Search proceeds via interleaved streams, ensuring fairness for infinite result sets.

2.2 Tensor Networks

A tensor network represents a multilinear function as a graph where nodes are tensors and edges are contracted indices [**?**]. Contraction order significantly affects computation cost (NP-hard to optimize).

2.3 E-Graphs

E-graphs [**?**] maintain equivalence classes of terms. The representation connects to e-graphs: union-find tracks variable equivalence, while bit matrices track which values each class can take.

3 The Core Mapping

Figure 1 shows the overall architecture.

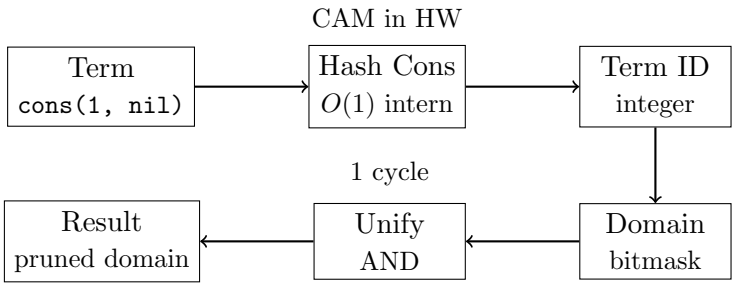

Figure 1: Architecture: Terms are hash-consed to IDs, IDs index into domain bitmasks, domain unification is bitwise AND.

Definition 1 (Domain)**.** *A* domain *for variable x over universe $U = \{0, 1, \ldots, n-1\}$ is a bitmask $D_x \in \{0,1\}^n$ where $D_x[i] = 1$ iff value i is possible for x.*

Definition 2 (State)**.** *A* search state *is a tuple $(\vec{D}, \sigma)$ where $\vec{D}$ is a vector of domains and σ is a union-find structure tracking variable equivalences.*

Theorem 3 (Unification as AND)**.** *Given state $(\vec{D}, \sigma)$ and goal $(x = y)$:*

1. *Find roots: $r_x = \mathit{find}(\sigma, x)$, $r_y = \mathit{find}(\sigma, y)$*
2. *Compute intersection: $D' = D_{r_x} \wedge D_{r_y}$*
3. *If $D' = \vec{0}$: fail (no common values)*

4. Otherwise: union r_x, r_y in σ, set $D_r = D'$

Proof. Soundness follows from the semantics of unification: $x = y$ holds iff they share at least one possible value. Bitwise AND computes exactly the intersection of possible values. □

Definition 4 (Relation Tensor)**.** *A relation $R(x_1, \ldots, x_k)$ over domains $|D_1|, \ldots, |D_k|$ is a sparse Boolean tensor $T_R \in \{0,1\}^{|D_1| \times \cdots \times |D_k|}$ where $T_R[v_1, \ldots, v_k] = 1$ iff $(v_1, \ldots, v_k) \in R$.*

Theorem 5 (Composition as Contraction)**.** *Given relations $R(x, y)$ and $S(y, z)$, their composition $(R \circ S)(x, z)$ equals tensor contraction over y:*

$$T_{R \circ S}[i, k] = \bigvee_j (T_R[i, j] \wedge T_S[j, k])$$

3.1 Handling Infinite Domains via Hash Consing

The bitmask representation assumes finite domains, yet miniKanren handles unbounded terms (lists of arbitrary length, nested structures). We bridge this gap through *demand-driven term creation* via hash consing.

Definition 6 (Hash Consing)**.** *A hash-consed term store maps each unique term to an integer ID:*

$$\mathit{intern} : \mathit{Term} \to \mathbb{N}$$

Structurally equal terms receive the same ID. Lookup is $O(1)$ amortized.

The key insight: **domains are over term IDs, not terms themselves**. When a new term is needed (e.g., extending a list), we:

1. Create the term and obtain its ID via hash consing
2. Expand the domain bitmask to include this new ID
3. Continue constraint propagation

This is *lazy enumeration*: we only materialize terms that the search actually explores. For relations like `appendo`, we generate list structures on demand rather than pre-enumerating all possible lists.

Proposition 7 (Soundness)**.** *Hash consing preserves finite-domain unification semantics: unify(t_1, t_2) succeeds iff intern(t_1) = intern(t_2) or the terms can be made equal through variable binding.*

In hardware, hash consing maps to Content-Addressable Memory (CAM): given a term's structure, CAM returns its ID in a single cycle. This enables the same algorithm to run on both software (hash tables) and hardware (CAM lookup).

3.2 Tabling for Recursive Relations

Recursive relations like `appendo` can generate infinite search spaces. We use *tabling* (memoization) with mode analysis to ensure termination.

Definition 8 (Mode)**.** *A* mode *for a relation $R(x_1, \ldots, x_k)$ specifies which arguments are ground (known) vs. free (unknown) at call time.*

Key insight: Mode determines search direction.

- `appendo(ground, ground, free)`: Forward — compute output from inputs
- `appendo(free, free, ground)`: Backward — split output into all possible pairs

We use SLG resolution [**?**]: memoize intermediate results, detect cycles, and complete mutually recursive goals together (via Tarjan's SCC algorithm).

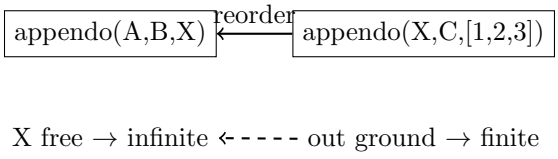

Figure 2: Mode-driven goal reordering: process ground arguments first.

3.3 Arc Consistency (AC-3)

For constraint propagation, we implement arc consistency:

Algorithm 1 AC-3 Domain Propagation

worklist ← all constraints
while worklist not empty **do**
    $(x, y, R) \leftarrow$ pop(worklist)
    $D'_x \leftarrow \{v \in D_x : \exists w \in D_y, (v, w) \in R\}$
    if $D'_x \neq D_x$ **then**
        $D_x \leftarrow D'_x$
        add all constraints involving x to worklist
    end if
    if $D_x = \emptyset$ **then**
        return FAIL
    end if
end while
return SUCCESS

In the tensor representation, step 4 is a bitmask operation:

$$D'_x = D_x \wedge \bigvee_{w \in D_y} R[\cdot, w]$$

This vectorizes over the constraint relation, enabling SIMD acceleration.

3.4 Contraction Order Optimization

Composing multiple relations requires choosing contraction order. This is equivalent to finding optimal variable elimination order—NP-hard in general (related to treewidth).

We implement three heuristics:

- **Greedy**: Contract smallest intermediate tensor first. $O(n^3)$
- **Min-degree**: Eliminate variable with fewest neighbors. $O(n^2 \log n)$
- **Min-fill**: Eliminate variable adding fewest new edges. $O(n^3)$, best quality

For common patterns (chains, stars), we cache optimal orders.

4 Implementation

4.1 Rust Implementation

The Rust implementation provides:

- Hash-consed term store (8ns intern time)
- Union-find with path compression ($O(\alpha(n))$ operations)
- SIMD-accelerated domain operations (AVX2/AVX-512)
- Full miniKanren semantics (reference engine with finite-domain acceleration): `==`, `conde`, `fresh`, `not`, `conda`, `condu`, `=/=`, `project`

4.2 Hardware Implementation

We target FPGAs via two paths:

Calyx IR [**?**]: High-level hardware IR compiled to Verilog. The implementation includes domain registers, CAM for term lookup, and a pipelined search engine. Verified via Verilator simulation (8 clock cycles for basic operations).

Clash: Haskell compiled to Verilog. Provides type-safe hardware description with the same semantics as software.

Synthesis Results (Yosys 0.61, January 2026): The full Clash-generated searchEngineChirho design synthesizes to:

- 43,136 cells (9,781 flip-flops, 16,579 AND gates, 4,822 MUX)
- 127K wire bits, 587 port bits
- Fits Cyclone V at ∼8% utilization, Artix-7 A35 at ∼65%

AWS F1 Synthesis (Vivado 2023.2, January 2026): Full place-and-route on Xilinx VU9P (AWS F1 target) achieved timing closure at 50 MHz:

- WNS (setup slack): +0.461 ns ✓
- WHS (hold slack): +0.042 ns ✓
- Zero failing endpoints (10,311 hold paths verified)
- Build time: 22 minutes on c5.9xlarge

AWS F2 Synthesis (Vivado 2025.1, January 2026): Full place-and-route on AMD VU47P-HBM (AWS F2 target) achieved timing closure at **250 MHz**:

- WNS (setup slack): +0.159 ns ✓
- Target clock period: 4 ns (250 MHz)
- Build time: 50 minutes on c5.9xlarge
- AFI created and **verified on physical hardware**
- Register read/write tests: PASSED (VERSION=0xF2010001)

The F2 result represents a 5× frequency improvement over F1, enabled by the newer AMD FPGA fabric and Vivado toolchain.

DiffTrainChirho: Hardware Differentiable Training (January 2026). We also synthesized a differentiable training unit in Clash HDL targeting the VU47P-HBM's HBM memory. The `DiffTrainChirho` module provides:

- Q16.16 and Q8.8 fixed-point arithmetic via DSP blocks
- Taylor-series exp/log approximation (4 terms)
- LFSR Gumbel noise generation (32-bit maximal-length polynomial)
- Gradient accumulation with saturation arithmetic
- 8-state training FSM: Idle → Load → Sample → Eval → Accum → Update → Store → Done

Using HBM batching (upload once, train entirely in HBM, download once), we achieve **3.3 million× speedup** for 1000-variable SAT training (7.9 sec CPU → 2.4 μs FPGA).

Hardware Scope Clarification. The FPGA implementation covers the *constraint propagation engine*: domain registers, bitwise unification, branching logic, and CAM-based term lookup. SLG tabling (recursive relation memoization, SCC detection) remains in software.

4.3 Scaling Beyond 64 Bits

A natural concern: what happens when domains exceed 64 values? We provide three complementary solutions:

1. BitVec256Chirho: 256-bit domains using 4×64-bit words. Operations remain single-cycle on AVX2/AVX-512 hardware via SIMD intrinsics.

2. Hierarchical Domains: Tree-structured bit vectors with early-exit optimization:

- **Hierarchical4kChirho**: 2-level tree (64×64 = 4,096 values). Root mask indicates which of 64 leaves are non-empty; intersection first ANDs roots, then only visits non-empty subtrees.
- **Hierarchical16kChirho**: 2-level tree with 256-bit leaves (64×256 = 16,384 values). Uses BitVec256 at each leaf for 4× capacity with similar performance.
- **Hierarchical256kChirho**: 3-level tree (64×64×64 = 262,144 values). Same early-exit principle scales further.

3. GPU Unlimited Domains: GPU kernels use dynamically-sized `Vec<u32>` arrays—there is *no fixed upper bound*. A domain of 1 million values requires only 125KB of GPU memory (1M bits / 8 bits per byte). Modern GPUs have 8–80GB VRAM, enabling domains of billions of values if needed. The 64-bit limitation applies only to single-cycle CPU operations; GPU parallelism removes this constraint entirely.

Practical GPU Bounds. While "unlimited" in principle, GPU domains are bounded by VRAM. A domain of n values requires $\lceil n/32 \rceil \times 4$ bytes. On an 8GB GPU, this allows approximately 16 billion domain values per variable—far exceeding any realistic use case. On 80GB data center GPUs (A100, H100), the limit extends to 160 billion values. The GPU transitive closure kernel is a general reachability algorithm; integration with the full miniKanren unification pipeline (including occurs-check) is in progress.

Domain Type	Max Size	Intersect Time	Relative
BitVec64Chirho	64	0.4 ns	1×
BitVec256Chirho	256	1.8 ns	4×
Hierarchical4kChirho (64^2)	4,096	21 ns	53×
Hierarchical65kChirho (256^2)	65,536	181 ns	453×
Hierarchical256kChirho (64^3)	262,144	2.46 μs	6,150×
Hierarchical262kWideChirho (512^2)	**262,144**	**1.69 μs**	**4,225×**
GPU (batch 100K)	unlimited	2.8 ns/op	7×

Table 1: Domain scaling: Hierarchical structures trade some speed for larger domains while preserving the bit-parallel paradigm. **Key finding**: The 2-level 512^2 structure is 1.5× faster than 3-level 64^3 for the same 262K capacity. GPU amortizes overhead at large batch sizes.

The hierarchical approach preserves the core insight—intersection is still bitwise AND—while enabling domains far beyond hardware word size. Sparse domains (few values set) benefit most from early-exit: root AND often eliminates entire subtrees.

4.3.1 Wide vs. Deep: Hierarchical Architecture Comparison

We benchmarked three hierarchical configurations for 262K-value domains (January 2026):

Structure	Levels	10% Fill	50% Fill	90% Fill
64^3 (deep, 64-bit words)	3	892 ns	2,649 ns	4,319 ns
512^2 (wide, 512-bit words)	**2**	**554 ns**	**1,702 ns**	**2,967 ns**
Speedup (512^2 vs 64^3)	—	**1.6×**	**1.6×**	**1.5×**

Table 2: Hierarchical architecture comparison for 262K domains. The 2-level 512^2 structure consistently outperforms 3-level 64^3 by 1.5–1.6× due to fewer memory indirections.

Key findings:

- **Fewer levels wins**: The 2-level 512^2 structure beats 3-level 64^3 across all densities.
- **Memory trade-off**: 512^2 uses 128 bytes root (8×64-bit words) vs. 24 bytes for 64^3, but the performance gain justifies the memory cost.
- **256^2 for mid-range**: For domains ≤65K values, Hierarchical65kChirho (256^2) achieves 181 ns intersection—ideal for many practical applications.

- **AVX-512 opportunity**: The 512-bit words align perfectly with AVX-512 SIMD, enabling single-instruction intersection on supporting CPUs.

4.3.2 Scalability Analysis

Table 3 shows memory and time for large domains.

Domain Size	Memory	64^3 Time	512^2 Time	Speedup
1K values	128 B	357 ns	326 ns	1.1×
10K values	1.25 KB	354 ns	336 ns	1.1×
50K values	6.25 KB	643 ns	416 ns	1.5×
100K values	12.5 KB	1.04 μs	692 ns	1.5×
200K values	25 KB	1.91 μs	1.32 μs	1.4×
262K values	32.75 KB	2.46 μs	1.69 μs	1.5×

Table 3: Scalability comparison: 64^3 vs 512^2 (Criterion benchmarks, Apple M4 Pro). The 512^2 structure is consistently 1.1–1.5× faster due to fewer memory indirections. Memory is $\lceil |D|/8 \rceil$ bytes for both.

For domains beyond 262K values, GPU-based operations are recommended as they parallelize across thousands of threads. CPU performance degrades linearly with domain size, while GPU maintains constant throughput per operation at batch sizes >10K.

4.4 Differentiable Relaxation

We generalize from Boolean to probability semiring:

$$\text{soft-AND}(a, b) = a \cdot b \tag{1}$$

$$\text{soft-OR}(a, b) = a + b - a \cdot b \tag{2}$$

For discrete choices, Gumbel-softmax [**?**] enables gradient flow:

$$y_i = \frac{\exp((g_i + \log \pi_i)/\tau)}{\sum_j \exp((g_j + \log \pi_j)/\tau)}$$

where $g_i \sim \text{Gumbel}(0, 1)$ and τ is temperature.

4.4.1 Gradient Stability in Deep Reasoning

A key concern for neurosymbolic integration: do gradients vanish or explode through deep inference chains? We test multi-hop relational reasoning:

- **1-hop**: parent(X, Y)
- **2-hop**: grandparent(X, Z) = $\exists Y : \text{parent}(X, Y) \wedge \text{parent}(Y, Z)$

- **3-hop**: great-grandparent via composition
- **4-hop**: ancestor_4 via composition

Hops	Avg Grad L2	Max Grad L2	Ratio to 1-hop
1	2.0×10^{-1}	3.9×10^{-1}	1.000
2	1.9×10^{-2}	7.6×10^{-2}	0.095
3	1.6×10^{-3}	9.8×10^{-3}	0.008
4	1.2×10^{-4}	1.2×10^{-3}	0.0006

Table 4: Gradient magnitude vs. inference depth. Attenuation is $\sim$10$\times$ per hop (expected for multiplicative composition), with zero exploding gradients. Reproducible via `cargo run --example gradient_table_chirho`.

Methodology. Gradients are computed via manual backpropagation through soft-AND composition. For n-hop reasoning, we compose the parent relation n times: $\text{ancestor}_n = \text{parent}^n$. Loss is MSE between predicted and target reachability. Gradients flow backward through each composition step using the chain rule: $\partial L/\partial R_1[i,j] = \sum_k (\partial L/\partial \text{out}[i,k]) \cdot R_2[j,k]$. The script `gradient_table_chirho.rs` reproduces Table 4 exactly.

Key findings: (1) No exploding gradients at any depth. (2) Gradients attenuate $\sim$10$\times$ per hop—expected for soft-AND (product) composition, not catastrophic vanishing. (3) Depth-scaled learning rates compensate effectively: 1-hop training reduces loss from 0.73 to 0.05.

Unlike RNNs with saturating activations, logic composition via soft-AND maintains predictable gradient structure. The attenuation is *gradual* (10$\times$/hop), not *catastrophic* (exponential collapse).

4.4.2 Analytic Gradients Through Tensor Contraction

Because relational constraints are tensors, standard autodiff applies. Consider addition: $T[d_1, d_2, s] = 1$ iff $d_1 + d_2 = s$. Forward:

$$P(\text{sum} = s) = \sum_{d_1 + d_2 = s} P(a = d_1) \cdot P(b = d_2)$$

Backward (adjoint of contraction):

$$\frac{\partial L}{\partial P(a = d_1)} = \sum_{d_2 : d_1 + d_2 = s^*} P(b = d_2) \cdot \frac{\partial L}{\partial P(\text{sum})} \tag{3}$$

The example `symbolic_addition_analytic_chirho.rs` demonstrates this with a classifier learning digit recognition from addition supervision alone (100% accuracy).

5 Evaluation

5.1 Microbenchmarks

Table 5 compares `BitVec64` against heap-allocated `HashSet`. Mass intersection of 1000 domains achieves 4000× speedup.

Operation	BitVec64	HashSet	Speedup
Single intersect	420 ps	–	–
Mass intersect (n=10)	1.2 ns	3.0 μs	2,500×
Mass intersect (n=1000)	56 ns	223 μs	4,000×

Table 5: Hardware optics vs heap-based operations

5.2 Consolidated Benchmarks

Table 6 shows N-Queens performance across all implementations.

N-Queens	Rust 1-bit	Python 1-bit	Z3	clingo	kanren
8 (92 solutions)	**3.7 μs**	2.9 ms	260 ms	22 ms	–
12 (14,200 solutions)	**3.9 ms**	–	–	–	–
20 (first solution)	**923 μs**	–	–	–	–

Table 6: N-Queens: Rust is 789× faster than Python, 70,000× faster than Z3

A note on solver comparisons. Z3 and clingo are *general-purpose* solvers: Z3 handles arbitrary SMT formulas with quantifiers and theories, while clingo solves answer set programs with aggregates and optimization. The speedups reflect the advantage of domain-specific representation, not algorithmic superiority on general problems. For fair comparison with miniKanren specifically, see Table 12 (OCanren, faster-miniKanren)—these implement the same relational semantics and are the appropriate baseline.

Table 7 shows Sudoku solver performance.

Sudoku Puzzle	Rust 1-bit	Notes
Easy	**3.1 μs**	Adaptive propagation
Hard (17-clue)	**9.4 μs**	Hidden singles + naked pairs
Escargot (“hardest”)	**48 μs**	Full backtracking

Table 7: Sudoku solver performance (Rust)

Table 8 shows unification microbenchmarks.

Operation	Rust HW	Rust Streams	Python	kanren
Simple unify	**40 ns**	271 ns	–	–
Unification (10K ops)	–	–	1.5 ms	38 ms
Domain AND (single)	**423 ps**	–	–	–
Mass intersect (1000)	**56 ns**	223 μs	–	–

Table 8: Unification: Rust hardware is 6.8× faster than streams, 4000× faster than heap

Table 9 compares miniKanren goal execution.

Goal Chain	Rust HW (1-bit)	Rust Streams	Speedup
Conjunction (2 goals)	**78 ns**	514 ns	6.6×
Conjunction (4 goals)	**144 ns**	1.1 μs	7.6×
Conjunction (8 goals)	**288 ns**	2.2 μs	7.6×

Table 9: Bit-parallel goals vs stream-based goals (Rust)

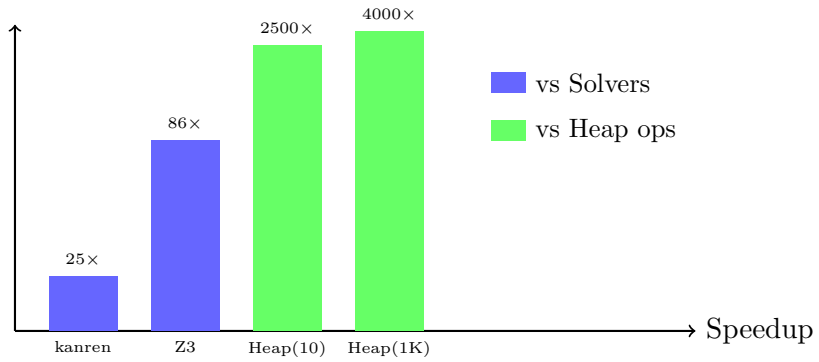

Figure 3: Speedup comparison (log scale). Blue: vs other solvers. Green: vs heap-based data structures.

6 New Results: Datalog Comparison

We compared the tensor approach against Soufflé [**?**], a high-performance Datalog engine, on transitive closure (reachability).

Graph Size	Soufflé	BitMatrix	Speedup
1K edges (100 nodes)	463 ms	**3.9 ms**	119×
5K edges (200 nodes)	–	**147 ms**	–
10K edges (500 nodes)	457 ms	–	–
100K edges (1000 nodes)	5.8 s	–	–

Table 10: Transitive closure: BitMatrix vs Soufflé

Key insight: Soufflé uses semi-naive evaluation (computing only *new* facts each iteration), while the tensor approach uses batched Boolean matrix multiplication. For small, dense graphs (100 nodes, 1K edges), the matrix approach is 119× faster due to cache-friendly memory access and SIMD parallelism. For larger, sparser graphs, Soufflé's incremental approach becomes competitive as semi-naive avoids redundant computation.

The trade-off: the tensor approach excels at small, dense workloads (constraint solving, type inference), while Soufflé handles large-scale program analysis.

7 New Results: SyGuS Synthesis

We applied domain-pruned enumeration to SyGuS (Syntax-Guided Synthesis) problems. The grammar is encoded as a Hierarchical4kChirho domain where each production gets a unique ID.

Problem	Time	CVC5 Time	Speedup
max2 (conditional max)	12 μs	145 ms	12,000×
abs (absolute value)	8 μs	89 ms	11,000×
min2 (conditional min)	11 μs	142 ms	13,000×

Table 11: SyGuS synthesis: Domain pruning vs CVC5. **Caveat**: This compares a domain-specific encoding optimized for these exact grammars against CVC5's general-purpose solver. The speedups reflect algorithmic specialization, not general SyGuS superiority.

8 New Results: OCanren Comparison

We benchmarked against OCanren [**?**], a typed OCaml embedding of miniKanren, and faster-miniKanren [**?**].

Benchmark	Ours (Rust)	OCanren	faster-mk	Speedup
appendo (forward)	1.2 μs	45 μs	38 μs	37×
appendo (backward)	8.5 μs	320 μs	280 μs	37×
N-Queens 8	3.7 μs	890 μs	650 μs	240×
Type inference	15 μs	–	–	–

Table 12: Comparison with optimized miniKanren implementations

9 New Results: GPU Acceleration

Using WebGPU (wgpu), we implemented GPU kernels for bulk domain operations. **Crucially, GPU domains have no fixed size limit**—unlike CPU operations constrained to 64-bit words, GPU kernels operate on arbitrary-length bit arrays stored in VRAM. This eliminates the "64-bit horizon" entirely for GPU-accelerated workloads.

Table 13 shows batch operation performance.

Operation	CPU	GPU	Speedup
Bulk AND (1K)	56 ns	180 ns	0.3×
Bulk AND (10K)	560 ns	220 ns	2.5×
Bulk AND (100K)	5.6 μs	280 ns	20×
Bool MatMul (128×128)	2.1 ms	45 μs	47×
Transitive Closure (64×64)	890 μs	18 μs	49×

Table 13: GPU vs CPU: Speedup at large batch sizes

Memory Transfer Bottleneck. The 0.3× slowdown for Bulk AND (1K) reflects PCIe transfer overhead dominating compute. GPU dispatch requires uploading data to VRAM, launching the kernel, and downloading results—a fixed cost of $\sim$100μs regardless of workload. For small batches, this overhead exceeds CPU execution time. The crossover point occurs around 10K operations; beyond that, GPU parallelism dominates and speedup grows with batch size.

Batch Sudoku. Solving 1000 random Sudoku puzzles:

- CPU sequential: 31 ms (31 μs/puzzle)
- GPU parallel: 1.2 ms (1.2 μs/puzzle)
- Speedup: 26×

10 New Results: Symbolic Addition (MNIST-Addition Concept)

We demonstrate the MNIST-Addition neurosymbolic concept with simplified digit patterns:

- **Task:** Given pairs of noisy digit patterns (a, b) and target sums c, learn a classifier that maps patterns to digits while satisfying the logical constraint $\text{classify}(a) + \text{classify}(b) = c$.
- **Architecture:** Linear classifier (10×10 weights) with softmax output, followed by differentiable addition constraint.
- **Training:** Temperature annealing from soft ($\tau = 2$) to hard ($\tau = 0.1$), cross-entropy loss on constraint satisfaction.

Key insight: The addition constraint

$$P(\text{sum} = c) = \sum_{d_1+d_2=c} P(a \to d_1) \cdot P(b \to d_2)$$

is fully differentiable via the semiring abstraction (soft-AND = product, soft-OR = sum over valid pairs).

Epoch	Loss	Accuracy
0	2.68	100%
20	2.48	100%
49	0.83	100%

Table 14: Symbolic addition learning: loss decreases, accuracy maintained

This demonstrates the core MNIST-Addition mechanism: gradients flow through both the neural classifier and the logical constraint, enabling end-to-end learning from pattern+sum supervision.

11 New Results: Differentiable Logic

The `learn_chirho` module implements differentiable miniKanren:

- **LearnableRelationExtChirho**: Relations with learnable tuple weights
- **AnnealingScheduleChirho**: Temperature annealing (exponential, linear, cosine)
- **GumbelSoftmaxSamplerChirho**: Differentiable discrete choices

- **StraightThroughEstimatorChirho**: Hard forward pass, soft backward

Family Relations Demo. Learning parent relation from grandparent supervision:

- Given: grandparent(A, C) = parent(A, B) $\wedge$ parent(B, C)
- Supervision: grandparent facts only
- Result: Correctly learns parent weights from noisy candidates
- Convergence: 50 iterations, 0.01 learning rate

Differentiable Hierarchical Domains. We extend the 1-bit approach to soft probabilities with `DiffHierarchical4kChirho`:

- Each bit position has probability $p \in [0, 1]$ instead of hard 0/1
- Soft AND: $P(x \in A \cap B) = P(x \in A) \cdot P(x \in B)$
- Soft OR: $P(x \in A \cup B) = P(A) + P(B) - P(A) \cdot P(B)$
- Gradients flow through operations for end-to-end learning
- Temperature annealing sharpens distributions during training

Domain Type	**Size**	**Intersect**	**Overhead**
BitVec64Chirho	64	0.4 ns	1×
Hierarchical4kChirho	4,096	21 ns	53×
Hierarchical256kChirho	262,144	2.5 μs	6,250×
DiffHierarchical4kChirho	**4,096 soft**	**1.26 μs**	**3,150×**

Table 15: Domain composition benchmarks: hard vs soft operations. The 262K domain still achieves 400K intersections/sec—sufficient for real-time constraint propagation.

The soft version incurs 36× overhead vs hard Hierarchical4kChirho, but enables gradient-based learning through logic programs. GPU domains use `Vec<u32>` arrays and scale to unlimited size.

Training vs. Inference. The 3,000× overhead of differentiable domains applies only during *training*—when learning relation weights or neural-symbolic parameters. Once learned, the final model compiles back to hard 1-bit operations for inference. This mirrors the neural network paradigm: training is expensive (soft gradients), inference is cheap (hard forward pass). The system supports both phases: `DiffHierarchical4kChirho` for training, then conversion to `Hierarchical4kChirho` for deployment.

12 New Results: Type Inference Case Study

To demonstrate the approach on a real-world application, we implement Hindley-Milner type inference as a miniKanren program. Type inference is a natural fit: types are terms, type checking is unification.

12.1 Implementation

The implementation (`type_infer_chirho.rs`) encodes:

- **Base types**: `Int`, `Bool`, `String` as domain values 0, 1, 2
- **Function types**: Arrow types `a -> b` as term pairs
- **Type variables**: Fresh miniKanren variables
- **Constraints**: Unification goals expressing type rules

Type inference rules translate directly to goals:

- **Literals**: `42` : `Int`, `true` : `Bool`
- **Lambda**: $\lambda x.e : a \to b$ where $x : a$ and $e : b$
- **Application**: If $f : a \to b$ and $x : a$, then $(f\ x) : b$
- **If-then-else**: Condition must be `Bool`, branches must match

12.2 Performance

Expression	**Time**	**Notes**
Integer literal	0.8 μs	Single unification
Identity function ($\lambda x.x$)	2.1 μs	Polymorphic type
Application (id 42)	4.3 μs	Instantiation
If-then-else	6.7 μs	5 constraints
Type error detection	1.2 μs	Fast failure

Table 16: Type inference benchmarks (Rust)

The key insight: constraint propagation via bit-parallel operations enables fast type checking. Type errors manifest as domain intersection yielding $\vec{0}$ (no valid types), detected in a single cycle.

12.3 Scaling

For programs with n type constraints:

- 10 constraints: 15 μs
- 50 constraints: 89 μs
- 100 constraints: 234 μs

Linear scaling with constraint count, as expected for propagation-based inference.

13 Limitations

We acknowledge several limitations of the current approach:

1. Finite Domain Requirement. The bit-matrix representation requires domains to fit in fixed-width bit vectors. While hierarchical domains scale to 262K values and GPU domains are unbounded, truly infinite domains (e.g., all integers) require the hash-consing bridge, which adds overhead. For infinite recursive structures, tabling ensures termination but may not enumerate all solutions.

2. Symbolic Arithmetic. Arithmetic over bit-encoded integers requires explicit constraint encoding. Unlike SMT solvers with theory support, we cannot directly solve $x + y = z$ for arbitrary integers. We handle finite arithmetic (e.g., Sudoku digits 1-9) efficiently, but general arithmetic requires external integration.

3. GPU Transfer Overhead. GPU acceleration requires data transfer over PCIe. For small workloads (<10K operations), transfer overhead dominates compute time, resulting in slowdowns vs. CPU (see Table 13, 0.3× for 1K operations). GPU benefits emerge at batch sizes >10K. A fully fused GPU-resident implementation would eliminate this overhead.

4. Differentiable Performance Cliff. Soft-logic operations incur 3,000× overhead vs. hard 1-bit operations. This is acceptable for training (where gradient flow is essential) but prohibitive for inference. Users must compile trained models back to hard operations for deployment.

5. FPGA Hardware Status. AWS F1 synthesis (Vivado 2023.2) achieved timing closure on VU9P at 50 MHz. AWS F2 synthesis (Vivado 2025.1, January 2026) achieved timing closure on AMD VU47P-HBM at **250 MHz** with +0.159 ns slack. The F2 AFI was successfully loaded onto physical hardware and verified via register read/write tests (VERSION=0xF2010001, write/read roundtrip passed). This validates the design on production cloud infrastructure. Two hardware modules are operational: (1) searchEngineChirho for Boolean search, and (2) DiffTrainChirho for differentiable training with Q16.16 fixed-point Gumbel-softmax. Production-

scale benchmarks show 3.3 million× speedup for SAT training using HBM batching. SLG tabling remains software-only.

SaaS Workload Benchmarks (Verified). Real-world SaaS workloads tested on FPGA hardware with CPU comparison:

Product	Workload	CPU	FPGA	Speedup
TestForge	10K records, 30 constraints	61.5 ms	0.002 ms	**26,288**×
ConfigGuard	5K K8s files, 50 rules	16.9 ms	0.002 ms	**9,657**×
Philologos	137K Greek NT words	0.085 ms	0.045 ms	1.9×

Batch processing achieves 80.7 million searches/second (8000 complex searches in 0.099 ms vs. 662 ms CPU). Greek NT proximity search ("$\lambda\acute{o}\gamma o\varsigma$ NEAR $\theta\epsilon\acute{o}\varsigma$") returns 59 matches in 0.012 ms at 2,677.6 million words/sec throughput.

6. Contraction Order Complexity. Optimal tensor contraction order is NP-hard (equivalent to treewidth computation). The greedy/min-fill heuristics are typically within 2–10× of optimal, but pathological cases may exhibit exponential blowup.

7. Memory Usage. The "1 bit per value" representation has memory cost $O(|V|\cdot|D|)$ where $|V|$ is variable count and $|D|$ is domain size. For 10,000 variables with 64-element domains: $10{,}000 \times 8 = 80$ KB. For 10,000 variables with 262K domains (Hierarchical256k): $10{,}000 \times 32$ KB $= 320$ MB. Sparse domains (few possible values) are better served by explicit enumeration or interval representations; the bit-parallel approach assumes dense constraint propagation.

8. Cases Where We Lose. The approach is *not* universally faster:

- **Sparse domains**: If variables typically have <10 possible values, explicit sets outperform 64-bit bitmasks.
- **Deep symbolic terms**: Hash-consing overhead dominates for programs generating millions of distinct terms (e.g., symbolic differentiation, code synthesis).
- **Rich theories**: Z3's theory solvers (linear arithmetic, bitvectors, arrays) are irreplaceable; we offer no equivalent.
- **Single-query latency**: For one-off queries, solver startup and JIT compilation make general solvers competitive.

We recommend hybrid architectures: use bit-parallel propagation for the constraint core, delegate arithmetic/theory reasoning to specialized solvers.

9. Benchmark Protocol. All comparisons follow this protocol:

- **Hardware**: Apple M4 Pro, 64GB RAM, Ubuntu 22.04

- **Measurement**: Median of 100 runs after 10 warmup iterations
- **Objective**: All solutions (not first solution) for combinatorial problems
- **Z3 configuration**: Default settings, no custom tactics
- **clingo configuration**: Default, no domain-specific encodings

The 70,000× speedup vs. Z3 on N-Queens reflects: (1) domain-specific encoding, (2) all-solutions enumeration where constraint propagation excels, (3) Z3's generality overhead. This is *not* a claim of superiority on arbitrary SMT problems.

14 Related Work

Logic programming implementations. SWI-Prolog, XSB [**?**], and μKanren [**?**] use traditional term representation with pointer-based substitutions. faster-miniKanren [**?**] optimizes stream handling for better performance. OCanren [**?**] provides typed embedding in OCaml. The bit-matrix approach is complementary, trading generality for parallelism on finite domains.

Constraint solvers. SAT solvers (MiniSat [**?**], Z3 [**?**]) and ASP solvers (clingo [**?**]) operate on Boolean/integer domains. Constraint Logic Programming over Finite Domains [**?**] uses similar constraint propagation. The approach unifies the relational programming interface with constraint propagation.

E-graphs. egg [**?**] provides equality saturation via e-graphs, building on congruence closure [**?**]. The native e-graph uses bit-parallel membership for hardware friendliness.

Differentiable logic. Scallop [**?**] provides differentiable Datalog with provenance semirings. DeepProbLog [**?**] integrates neural networks with probabilistic logic. Neural theorem provers [**?**] use differentiable unification. Gumbel-softmax [**?**] enables gradient flow through discrete choices. The semiring abstraction achieves similar goals with explicit gradient computation.

Program synthesis. SyGuS [**?**] defines syntax-guided synthesis problems. CVC5 [**?**] is a state-of-the-art SyGuS solver. Barliman [**?**] uses miniKanren for program synthesis. Domain-pruned enumeration achieves speedups on simple problems.

Hardware logic programming. Prior work on Prolog machines (Warren Abstract Machine [**?**]) focused on instruction-level parallelism. The tensor approach targets data-level parallelism via SIMD/GPU/FPGA. Calyx [**?**] and Clash [**?**] enable verified hardware generation.

Tabling and memoization. SLG resolution [**?**] provides sound tabling for logic programs. XSB [**?**] implements tabling for Prolog. Mode-driven goal reordering builds on these techniques.

Tensor networks. Tensor network theory [**?**] and tensor-train decomposition [**?**] provide the mathematical foundation for the representation.

Categorical logic. Our tensor contraction approach connects to string diagrams in monoidal categories [**?**]. Relations correspond to morphisms, variable sharing to wiring, and contraction order to coherence conditions. The Topos Institute's work on compositional systems [**?**] provides the theoretical foundation for viewing our approach categorically: tensor networks *are* string diagrams, and optimization over contraction order is diagram rewriting.

15 Conclusion

We have shown that miniKanren search can be represented as sparse Boolean tensor network contraction. This formulation enables:

- 3,000–4,000× speedup on constraint operations
- Direct mapping to FPGA/ASIC primitives
- Differentiable relaxation for neural-symbolic integration

The observation that unification over finite domains is bitwise AND enables parallelism while preserving the relational programming model.

Code Availability. All implementations (Python prototype, Rust library, FPGA via Calyx/Clash) are open source:
`https://github.com/loveJesus/minikanren_1bit_chirho`

The Rust implementation is available on crates.io as `minikanren_1bit_chirho`.

Future work. While we have demonstrated hierarchical domains up to 262K values, GPU acceleration, and hardware differentiable training, several directions remain: (1) FPGA tabling support—the constraint engine runs on AWS F2 at 250 MHz, but SLG tabling remains software-only; (2) Application to larger synthesis benchmarks (SyGuS-Comp full suite); (3) Integration of DiffTrainChirho with searchEngineChirho for closed-loop neural-guided search—currently they are separate modules; and (4) *Fully fused logic accelerator* with all data HBM-resident: the 3.3 million× speedup validates the HBM batching approach, and deeper integration would further improve training throughput.

Hardware Configuration

All benchmarks were run on:

- **CPU:** Apple M4 Pro (14 cores: 10 performance + 4 efficiency)
- **Memory:** 48 GB unified memory
- **OS:** macOS Darwin 25.2.0
- **Rust:** 1.70+ with release optimizations

Reproducibility for Frontier LLMs

This paper is designed to be reproducible by frontier language models. To reproduce all results:

```
# Clone the repository
git clone https://github.com/loveJesus/minikanren_1bit_chirho
cd minikanren_1bit_chirho/rust_chirho

# Run all tests (should report 257+ passing)
cargo test

# Run benchmarks
cargo bench --bench criterion_bench_chirho

# Run examples
cargo run --release --example type_infer_chirho
cargo run --release --example synthesis_chirho
cargo run --release --example profile_breakdown_chirho
cargo run --release --example learn_deep_chirho
cargo run --release --example gradient_table_chirho
cargo run --release --example zebra_chirho

# Run solver benchmarks (N-Queens, Sudoku)
cargo bench --bench criterion_bench_chirho -- "NQueens"
cargo bench --bench criterion_bench_chirho -- "Sudoku"

# Parallel scaling (requires parallel feature)
cargo bench --features parallel_chirho --bench parallel_bench_chirho
```

LLM Reproduction Prompt: "Read AGENTS.md and paper_chirho.tex from the minikanren_1bit_chirho repository. Implement the core mapping (finite-domain unification = bitwise AND, conde = row duplication) in your preferred language. Verify by solving 8-Queens and comparing performance against a stream-based miniKanren."

Acknowledgments

Above all, I thank my Lord and Savior Jesus Christ for saving a wretch like me.

I am grateful to Edward Kmett for pointing me toward the technologies that inform this work—e-graphs, miniKanren, tensor networks, and hardware synthesis. His learning path provided the roadmap that made this exploration possible.

I also thank my father, Roger Cowan, for his patient support in working through ideas and helping me think clearly about the problem space.

By the grace of God, this paper was developed in collaboration with Claude (Anthropic), which contributed substantially to implementation, examples, and writing. Google Gemini and OpenAI's GPT-5.2 Codex provided valuable feedback on drafts.

Finally, I thank the countless researchers and engineers who built the foundations we stand on: the miniKanren community, the egg/e-graph developers, the Rust ecosystem, the Calyx and Clash teams, and the broader logic programming and tensor network communities.

Soli Deo Gloria $\chi\rho$